SIT720 - Machine Learning

9.1 PASS TASK

Decision Tree and Ensemble Methods

Field	Details
Student	Zenith Anthony
Student ID	s225534396
Dataset	UCI Car Evaluation
Target classes	Unacceptable, Acceptable, Good
Evaluation design	80:20 stratified split; 3-fold stratified CV on training data

Corrected report incorporating executed notebook results

Part A - Lesson Review and Reflection

A1. Key Learning from Week 8: KNN and Decision Trees

Concept 1 - Gini Index and tree splitting. The Gini Index measures how mixed the classes are within a node. A value of zero means the node contains one class only, while larger values indicate greater class mixture. During tree learning, candidate splits are assessed by the reduction in weighted Gini impurity. The split with the strongest reduction creates purer child nodes and is therefore more useful for classification. In the car dataset, a split on passenger capacity can isolate cars that carry only two people; in the fitted tree, this branch is completely Unacceptable. The impurity reduction gives a numerical measure of how much class separation the split creates.

Concept 2 - Tree depth and model complexity. Tree depth controls how many sequential decisions a tree can make. A shallow tree may be easy to interpret but can underfit because it cannot represent enough interactions. A deep tree can fit complicated combinations of safety, persons, buying price, maintenance cost and luggage size, but it may have higher variance and fit training-specific details. Comparing training performance with cross-validation performance across several depths shows the bias-variance trade-off. The selected depth should be based on validation evidence rather than the highest test score, because the test set should remain untouched until final evaluation.

A2. Key Learning from Week 9: Ensemble Learning

Concept 1 - Random Forest and bagging. Bagging trains multiple decision trees on bootstrap samples and combines their predictions by voting. Random Forest adds feature randomness at each split, which makes the individual trees less correlated. Combining many different trees reduces variance and usually produces a more stable classifier than relying on one tree. Out-of-bag observations, which are not selected in a tree's bootstrap sample, can also provide an internal estimate of predictive performance. In this task, Random Forest is tuned for the number of trees, maximum depth and number of features considered at each split.

Concept 2 - AdaBoost. AdaBoost creates a strong classifier from a sequence of weak learners, usually shallow decision trees. After each learner is fitted, the algorithm gives greater emphasis to observations that were previously misclassified. Later learners therefore focus more strongly on difficult cases, and their predictions are combined into the final decision. The learning rate controls how strongly each new learner contributes, while the number of estimators controls the number of boosting rounds. This can work well when successive learners correct useful residual errors, although the experiment in Part B shows that boosting is not always the strongest method for every dataset.

A3. Applying Decision Trees and Ensemble Learning to Data Analytics

Decision trees are useful when predictions can be expressed as understandable rules. In vehicle evaluation, a tree can classify a car from safety, carrying capacity, buying price, maintenance cost, luggage space and doors. An analyst can trace the path from the root to a leaf and explain which conditions caused the prediction. This makes trees useful when interpretability matters. Ensemble methods can improve the basic approach: bagging reduces the sensitivity of one tree to a particular training sample, while boosting focuses sequentially on difficult observations. A practical application is an online vehicle screening or recommendation system. A single tree could support transparent explanations, while an ensemble could provide the production prediction. The final choice should be based on validation evidence and class-wise performance rather than accuracy alone, particularly when the target classes are imbalanced.

Part B - Problem Solving Practical Task

1. Data Exploration and Pre-processing

The UCI Car Evaluation dataset contains 1,728 observations and six categorical input features: buying, maint, doors, persons, lug_boot and safety. The original target has four classes. Following the task specification, good and vgood were merged into a single Good class, giving a three-class prediction problem.

Class	Count	Percentage
Unacceptable	1210	70.02%
Acceptable	384	22.22%
Good	134	7.75%

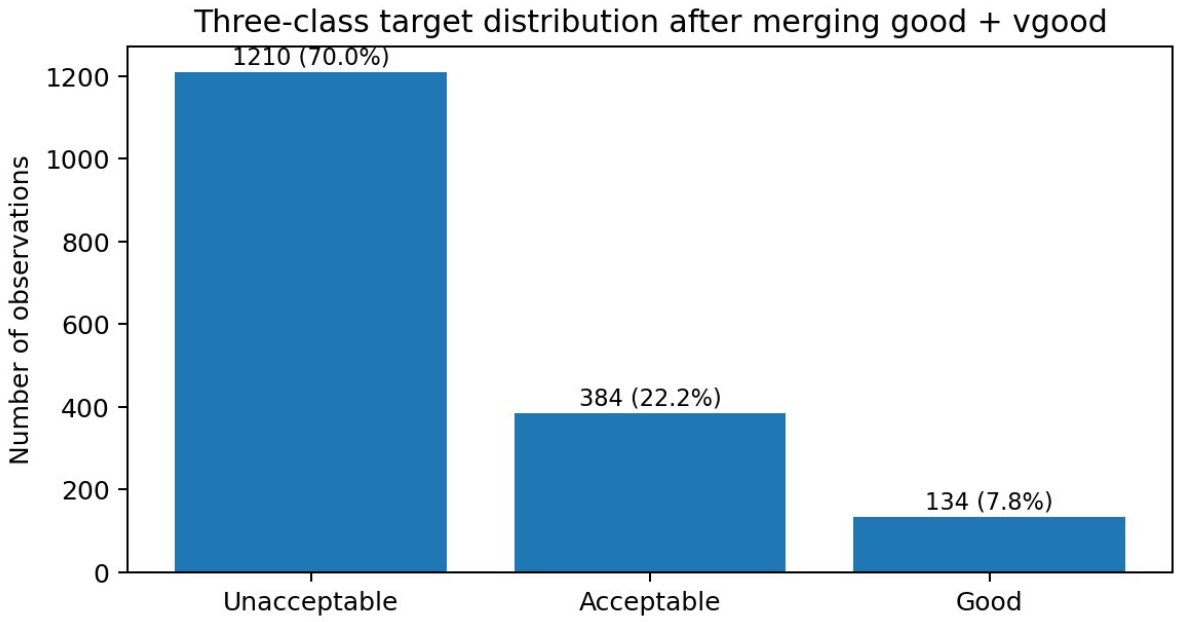


Figure 1. Three-class target distribution after merging good and vgood.

The target is imbalanced because Unacceptable accounts for about 70% of the data while Good accounts for only 7.75%. For this reason, macro precision, macro recall and macro F1 are reported together with accuracy. Macro averaging gives each class equal importance.

Encoding and split. The six predictors have meaningful ordered categories, so ordinal encoding was applied with explicit category orders (for example, safety: low < med < high; persons: 2 < 4 < more). This preserves the natural order without creating arbitrary label codes. The data were then divided using an 80:20 stratified train-test split. The encoder was fitted on the training data only, avoiding leakage. The resulting sets contain 1,382 training observations and 346 test observations.

2. Baseline Decision Tree Model

A full DecisionTreeClassifier was fitted with Gini impurity and no maximum-depth restriction. The resulting tree had depth 12 and 72 leaves. The untouched test set was used for the following final baseline metrics.

Metric	Test result
Accuracy	0.9913
Macro precision	0.9718

Macro recall	0.9790
Macro F1-score	0.9753

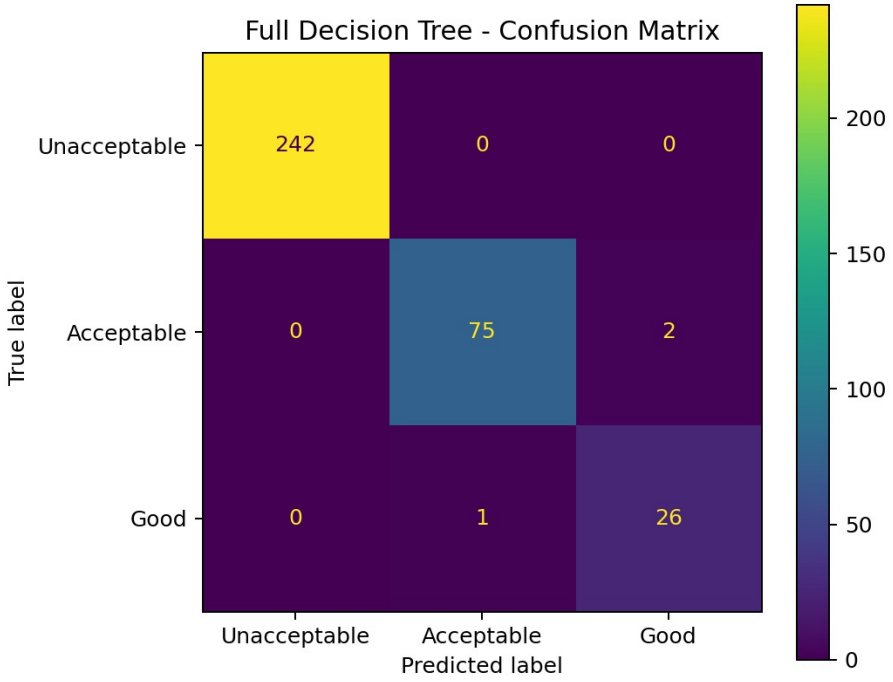


Figure 2. Baseline full Decision Tree confusion matrix.

The Unacceptable class was classified perfectly in this split (242/242). For Acceptable, 75 of 77 cases were correct and two were predicted as Good. For Good, 26 of 27 were correct and one was predicted as Acceptable. Good was the most difficult class by F1-score (0.9455) and precision (0.9286), which is consistent with it being the smallest class.

3. Decision Tree Interpretation

The root feature was persons. With the explicit ordinal encoding, persons values are 2=0, 4=1 and more=2, so the first split persons <= 0.50 separates cars with capacity for only two people from all cars carrying four or more people. The left child is completely pure and predicts Unacceptable.

Tree interpretation item	Observed value
Root feature	persons
First split condition	persons <= 0.50 (equivalent to persons = 2)
Root Gini impurity	0.454051
Weighted child Gini	0.381667
Impurity reduction	0.072383

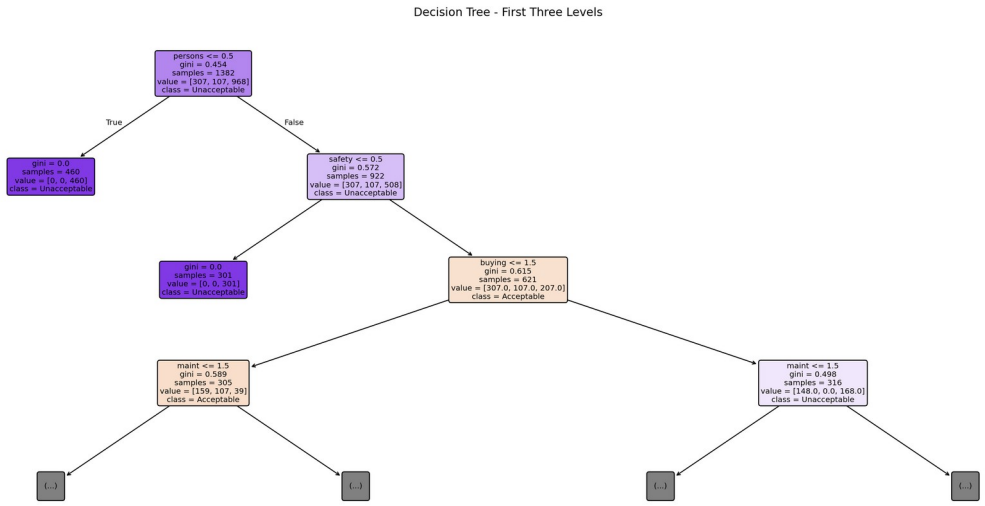


Figure 3. First three levels of the fitted full Decision Tree.

The next major split uses safety. When persons > 2 but safety is low, the tree again predicts Unacceptable. For cars with more passenger capacity and safety above low, later rules use buying cost, maintenance cost and luggage capacity. These paths illustrate the interpretability of Decision Trees: each prediction can be translated into a sequence of understandable if-then conditions.

4. Tree Complexity and Bias-Variance Trade-off

Tree depths from 1 to 15 were evaluated. Depth was selected using 3-fold stratified cross-validation on the training set with macro F1 as the selection metric. The test scores shown below are reported for analysis but were not used to choose the depth.

Depth	Train acc.	Test acc.	Train macro F1	CV macro F1
1	0.7004	0.6994	0.2746	0.2746
4	0.8799	0.8844	0.8141	0.8139
6	0.9392	0.9422	0.8931	0.8904
8	0.9805	0.9827	0.9617	0.9186
10	0.9964	0.9884	0.9936	0.9378
12	1.0000	0.9913	1.0000	0.9377
15	1.0000	0.9913	1.0000	0.9377

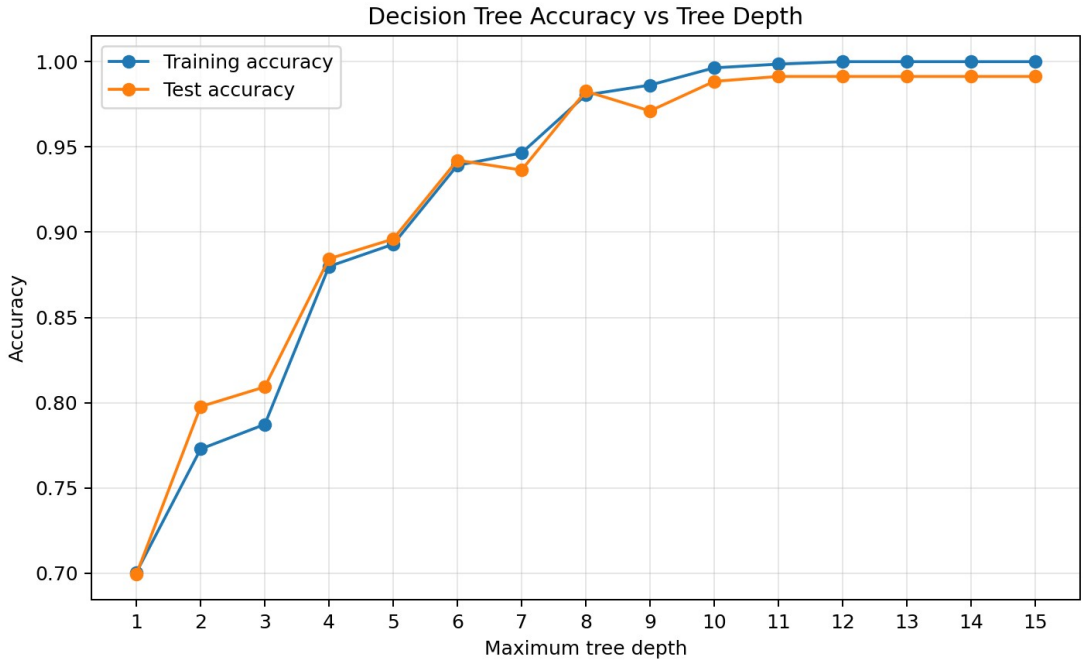


Figure 4. Training and test accuracy across Decision Tree depth.

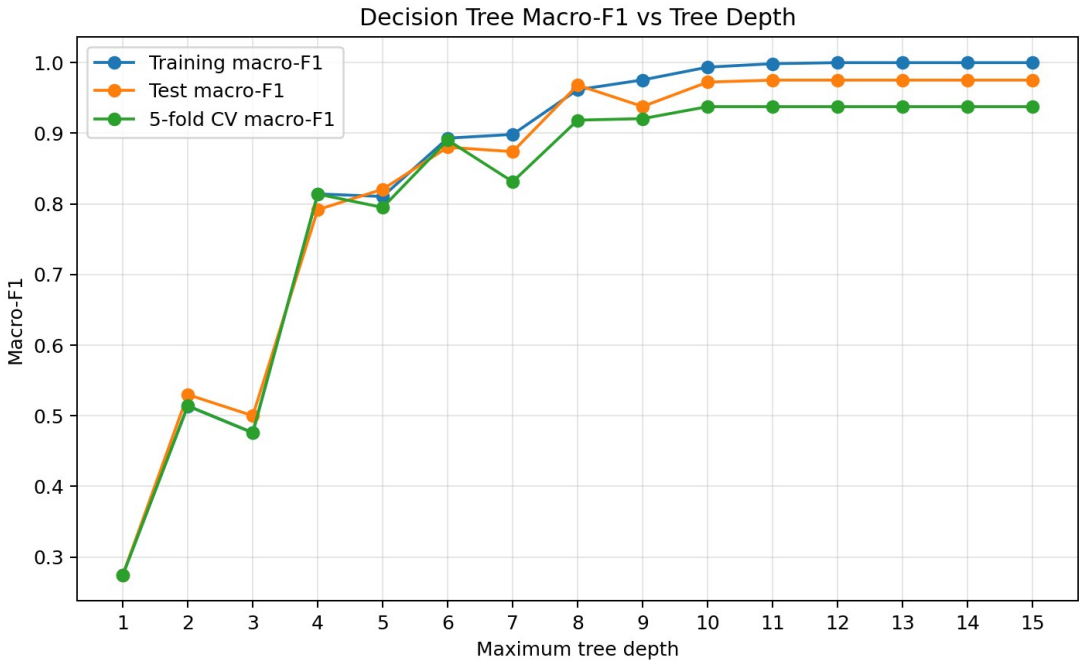


Figure 5. Training, test and cross-validation macro F1 across Decision Tree depth.

Depth 1 clearly underfits: training accuracy is only 0.7004 and macro F1 is 0.2746, indicating high bias. Performance rises strongly as the tree becomes more expressive. The highest cross-validation macro F1 was 0.9378 at depth 10, so depth 10 was selected before examining the final test result. At depth 10, training accuracy was 0.9964 and training macro F1 was 0.9936; the final test accuracy was 0.9884 and test macro F1 was 0.9725. At depth 12 and above, training performance reaches 1.0000 while CV macro F1 does not improve (about 0.9377), showing that extra complexity offers no validation benefit and increases variance risk.

5. Random Forest: Bagging Ensemble Learning

Random Forest hyperparameters were tuned using the same 3-fold stratified training cross-validation and macro F1. The search varied n_estimators, max_depth and max_features. The best configuration used 100 trees, max_depth=10 and max_features=None.

Random Forest quantity	Result
Best n_estimators	100
Best max_depth	10
Best max_features	None
Training CV macro F1	0.9565
OOB score	0.9834
OOB error	0.0166
Final test accuracy	0.9884
Final test macro F1	0.9725

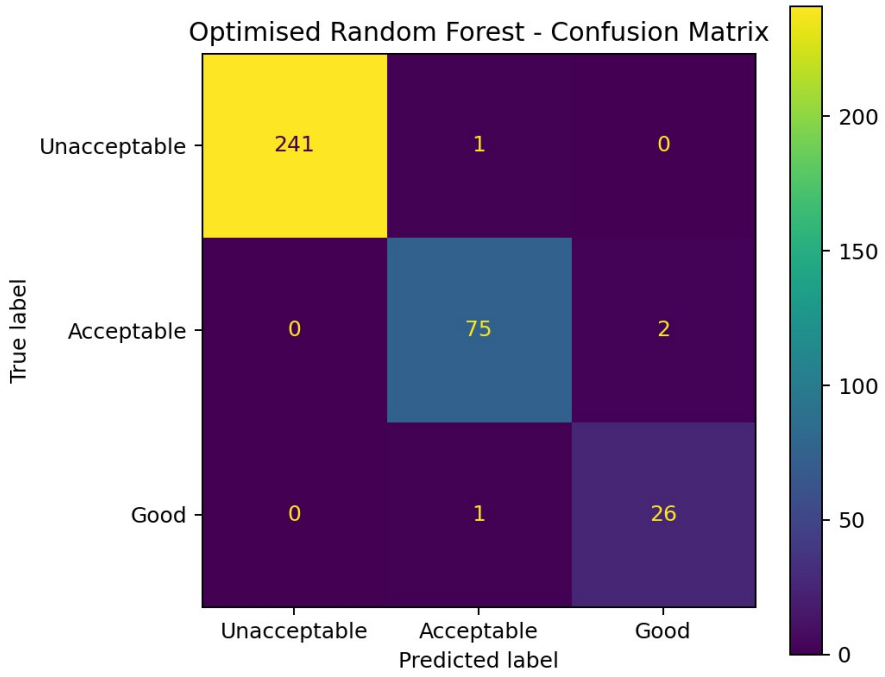


Figure 6. Optimised Random Forest confusion matrix.

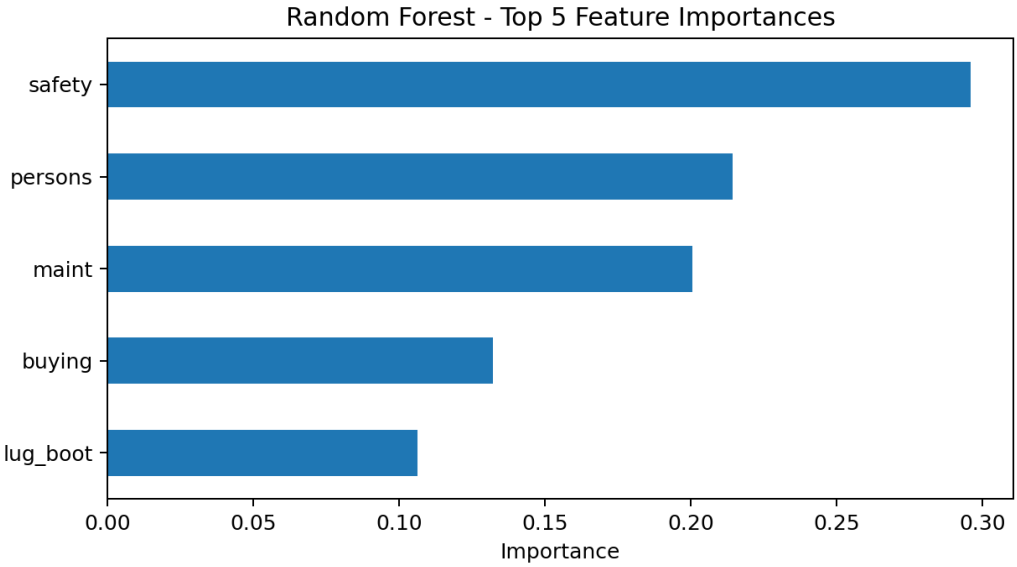


Figure 7. Top five Random Forest feature importances.

The five most important features were safety (0.2959), persons (0.2144), maint (0.2005), buying (0.1320) and lug_boot (0.1062); doors had the lowest importance (0.0511). Safety and passenger capacity are plausible leading features because low safety or a two-person capacity strongly constrain acceptability in the learned tree. The Random Forest and the depth-10 Decision Tree produced the same final test accuracy and macro F1 in this split, so the test set does not show a performance gain for the forest. However, Random Forest had the stronger training-set CV macro F1 (0.9565 versus 0.9378) and a high OOB score, so it is preferred based on model-selection evidence rather than the test tie.

6. Boosting: Building Strong Learners from Weak Learners

AdaBoost used depth-1 Decision Trees as weak learners. A training-set grid search tuned the number of estimators and learning rate. The best setting was n_estimators=200 and learning_rate=1.0.

AdaBoost quantity	Result
Best n_estimators	200
Best learning_rate	1.0
Training CV macro F1	0.8064
Final test accuracy	0.8786
Final test macro F1	0.8348

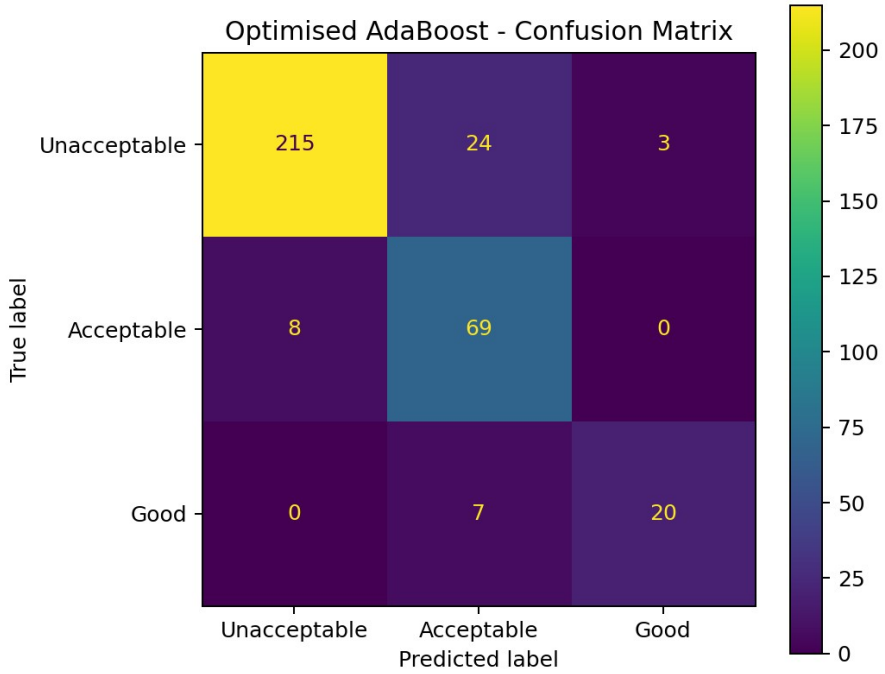


Figure 8. Optimised AdaBoost confusion matrix.

AdaBoost performed substantially below the tree-based bagging approach. Its Good-class recall was 0.7407 and Acceptable precision was 0.6900. This indicates that sequentially reweighting the difficult cases did not match the structured categorical decision boundaries as effectively as the tree/forest models under this configuration. Bagging is therefore more suitable for this experiment: Random Forest achieved CV macro F1 of 0.9565 compared with 0.8064 for AdaBoost.

Final Model Comparison and Preferred Approach

The preferred approach was selected from training-set cross-validation, before using the test set for final reporting. Random Forest had the highest CV macro F1, so it was selected. The final test results are reported once for comparison.

Model	Training CV macro F1	Final test accuracy	Final test macro F1
Decision Tree (depth 10)	0.9378	0.9884	0.9725
Random Forest	0.9565	0.9884	0.9725
AdaBoost	0.8064	0.8786	0.8348

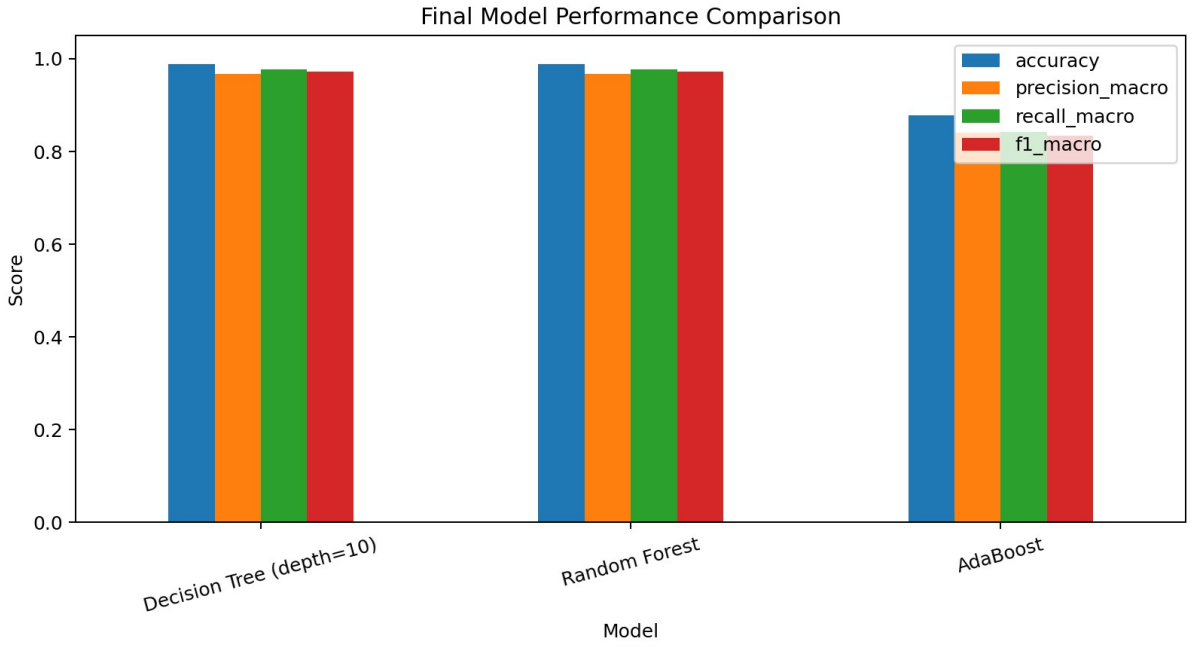


Figure 9. Final untouched test-set performance comparison.

Conclusion

The experiment shows that a single Decision Tree can model the Car Evaluation problem very accurately while remaining interpretable. A depth-10 tree was selected using training cross-validation, avoiding test-set-driven model selection. Random Forest produced the strongest cross-validation evidence (macro F1 0.9565) and was therefore selected as the preferred approach. On the untouched test set, Random Forest and the selected Decision Tree tied at 0.9884 accuracy and 0.9725 macro F1, while AdaBoost was lower at 0.8786 accuracy and 0.8348 macro F1. The class imbalance makes macro metrics important, particularly for Good. Overall, the evidence supports bagging with Random Forest for this dataset, while retaining the Decision Tree as a useful interpretable model.

GenAI Acknowledgement

Generative AI (ChatGPT) was used to assist with planning the analysis, clarifying machine-learning concepts, debugging and editing Python code, and improving report wording. The notebook was executed after the feedback was received, the numerical outputs and figures were checked, and the report was updated using those executed results. The student should review the final wording and ensure it accurately represents their own understanding before submission.

References

[1] M. Bohanec, "Car Evaluation," UCI Machine Learning Repository, 1988, doi: 10.24432/C5JP48.

[2] M. Bohanec and V. Rajkovic, "Knowledge acquisition and explanation for multi-attribute decision making," 8th International Workshop on Expert Systems and their Applications, Avignon, France, 1988, pp. 59-78.

[3] scikit-learn documentation: DecisionTreeClassifier, RandomForestClassifier, AdaBoostClassifier, GridSearchCV and classification metrics.